Lab 2: Single-Cycle RISC-V Microarchitecture Implementation in SystemVerilog HDL

Assigned: Monday 2/17; Due **Monday 3/10** (midnight)

Instructor: James E. Stine, Jr.

1. Introduction

In this lab, you will implement a RISC-V processor using a hardware description language (SystemVerilog). The microarchitecture of the RISC-V RV32I machine is single-cycle, a simple microarchitecture that we covered in class.

This lab's objective is to see the issues related to its implementation in hardware for the single-cycle RISC-V processor. For this laboratory, we will use the SystemVerilog Hardware Descriptive Language (HDL) modeling language. Hopefully, this laboratory will also allow you to see the choices for the RISC-V Instruction Set Architecture and how they make hardware better or worse. For full credit, you should achieve the following in this lab.

- Implement a single-cycle RISC-V RV32I machine in SystemVerilog for the RV32I instruction set.
- Ensure that your implementation is correct by simulating it and verifying the register dumps of the test inputs.
- Implement your design on the National Instruments Elvis III and DSDB boards and the onboard SDRAM using the DDR3 built-in controller.

2. Specifications of the RISC-V RV32I Machine

2.1 Instruction Set Architecture

The machine should support all of the following RISC-V RV32I instructions

~~add~~	~~addi~~	~~andi~~	~~and~~	auipc
~~beq~~	bge	bgeu	blt	bltu
bne	jalr	~~jal~~	lb	lbu
lh	lhu	lui	~~lw~~	~~or~~
~~ori~~	sb	sh	sll	slli
~~slt~~	~~slti~~	sltiu	sltu	sra
srai	srl	srli	~~sub~~	~~sw~~
xor	xori			

2.2 PC Fetch

Similar to Lab 1, the PC register directly reflects the `PC + 4` offset specified by RISC-V. Therefore, if the last instruction executed is at address `0x8000_0020`, the PC dumped will have the value `0x8000_0024` (next instruction to execute). However, to ensure correctness when using PC, every instruction that uses PC should include the +4 offset (e.g., JAL). The only exception to this rule is when performing a link or branch. In this case, you should always store the address of the next instruction.

2.3 Microarchitecture

The machine has a single-cycle microarchitecture: every instruction takes exactly one cycle to execute. Aside from correctness (as defined by the architectural specifications), this is the only constraint that we are placing on the machine's microarchitecture. As long as these two constraints are satisfied (i.e., correctness and

single-cycle operation), you are free to implement the microarchitecture in anyway you want. To guide you along the way, we provide an description of the single-cycle microarchitecture that has some of the instructions implemented (13 instructions[1]). A testbench is also given along with testbench that runs the sample code through. You can, of course, run your code from lab1 or other hand built code. Some things to keep in mind related to this implementation:

- The architectural state of the machine (excluding memory) is stored in registers: general-purpose registers.
- There is a global `logic` definition called the "clock (clk)" that is connected to all the registers. The clock synchronizes all events for update
- When a register sees a rising edge on the clock, the register captures the instantaneous "snapshot" of the values on its input. From then on, the register holds the captured values and feeds them to its output. (i.e., the clock edge basically activates a write).
- The output from the register(s) are fed into a combinational circuit consisting of logic gates (e.g., ADD) to instigate the control. In turn, the output from the logic gates are fed back as input to the register(s) that only change on the postive edge of the clock.
- At the next rising edge on the clock, the register again captures the values on its input.
- At each rising edge, the execution of an instruction is initiated. At the next rising edge, the values stored in all the registers (i.e., the architectural state) should be updated in such a way that the instruction can be considered to have correctly executed.
- The SystemVerilog model given to you utilizes a Princeton architecture. The code is loaded through the DO file at the beginning of simulation. Please check the example code to understand how this works.

The odd thing about this type of design, as mentioned in class, is that current instructions really do not update the state of the architecture until the next clock cycle.

3. Behavioral Memory for Simulation

As mentioned in class and in the textbook, the memory is stored as bytes. For this lab, the memory module handles the memory as a Harvard architecture. That is, there is one memory where the instructions are stored and the other where data is stored. Also, I made sure it stored data in little-endian mode and the SystemVerilog DO file loads the program. If you run the sample program through correctly, as stated within the assembly file, it should write the value of 7 to address 100. You can check the result of other programs by using your simulator from Lab 1.

You can modify the line in the DO file (`riscv_single.do`) to run the simulation. simulation and puts the program at location `0x0`. You should not see a difference as the `flopenr` module within `riscv_single.sv` loads the starting memory location into the PC upon a reset.

4. Synthesis and Implementation on the National Instruments ELVIS III board

For this lab, we will also implement our architecture into a real device with real memory. The board we plan on using is the National Instruments Virtual Implementation Suite or NI ELVIS III board. In addition to the ELVIS III board, we will attach an add-on device to give us use of a Xilinx Zynq-7Z020 FPGA. For this, we will be using the National Instruments Digital System Development Board (DSDB). The DSDB board is also populated with two Micron DDR3 memory components to give a single rank, 32-bit wide interface with a total of 512 MB of data memory.

[1] striked through in Section 2.1

The DSDB board plugs into the ELVIS III and make sure you have this board already on your desk. If not, please contact your TA. The board looks like the PCB in Figure 1. The FPGA is labeled 18 and has a heat sink on it, whereas, our SDRAM memory is labeled 19 in Figure 1. The board plugs into the ELVIS III board through the connector labeled 11. There are other pins and items that we will utilize throughout the labs to help us with debugging. Additional documentation on the board including a schematic can be found in the `datasheets` directory of your repository (i.e., files you download from the repository).

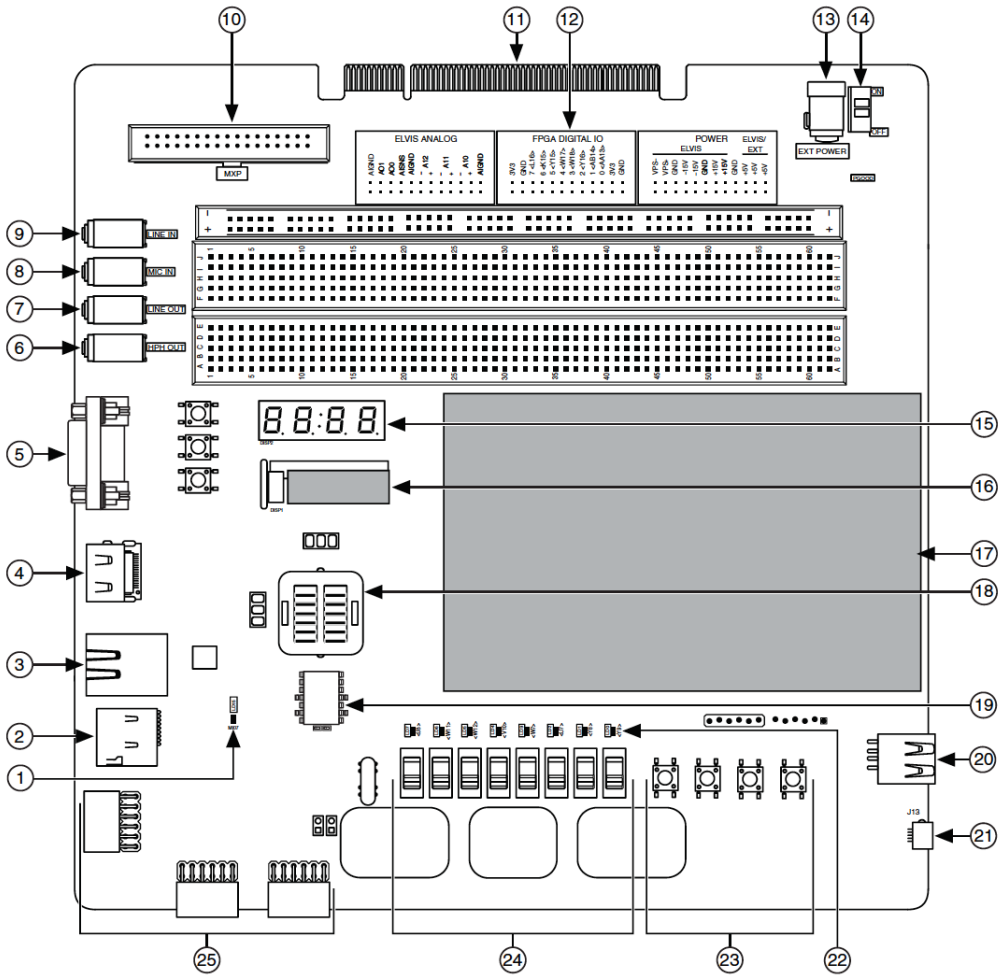

Figure 1: National Instruments Digital System Develpment Board (DSDB).

To work with the FPGA we will be using the Vivado Design Suite, which will handle synthesis, implementation, and also give us access to some debugging tools. However, it should be mentioned that, even with debugging tools, it is *significantly harder* to identify problems in your SystemVerilog on an FPGA, let alone fix them, if you do not already have things working in simulations like Questa/ModelSim. Therefore, it is extremely important you use any resource that is available to help in debugging. You will be provided with a Vivado project to get you started that already has most of the configuration details set up automatically. See the `fpga_guide.pdf` for information on how to implement the design on the DSDB board.

The top-level design in Vivado utilizes a fabric or interconnection matrix to connect the programmable logic (PL) to parts of the chip we will use to communicate with the memory. This interface in AMD is called AXI or Advanced eXtensible Interface. The AXI interface is based on ARM's AMBA 3.0 open standard and is a specific Intellectual Property (IP) block on the FPGA. The AMBA standard was originally developed by ARM for use in microcontrollers, with the first version being released in 1996 and is now used in many devices that deal with external devices [1]. Basically, the AXI interface provides a methodology for arbiting

connections on its bus structure (i.e., making sure two signals are not fighting each other). Again, do not attempt to implement your design until you are sure your hardware can simulate the `test_hw.S` program properly!

4.1 Memory Interfacing

One of the most challenging elements in computer architecture is utilizing memory. Although memory is typically a different chip or integrate circuit on most designs, its crucial to the operation of any software. Good performance always comes from good memory design as well as getting the data to the processor quickly. To accomplish this, most designers utilize DRAM as its relatively fast, but most importatly can be integrated with large memory spaces (i.e., in the GB). Although DRAM is useful, it involves complicated memory interfaces to make sure the data can be stored accurately and quickly on the DRAM device. This is a tradeoff engineers made to make DRAM big - that is, they sacrified the complexity of the circuit for the size of DRAM that can be created.

To make the data gets to the processor correctly, a memory controller is usually used. Most designs today use a fast memory controlled called a Double Data Rate (DDR) synchronous inteface. This memory controller is crucial to making the Synchronous DRAM works correctly. It speeds up the transfer by utilizing both edges of the clock. Fortunately, there is a DDR3 controller on the FPGA, so we will use this IP that is already there through the AXI interface via a memory mapped address. The Double Data Rate (DDR) memory controller consists of three major modules: a core memory controller and scheduler (DDRC), an AXI memory port interface (DDRI) and a digital PHY and controller (DDRP) [1]. Although the DDR controller allows the transfer of data to/from the processor easily, it is an asynchronous protocol in that it has to wait for the memory to respond that it is complete.

This means you will have to utilize a specific protocol on your RISC-V RV32I single-cycle architecture to make sure it works. The protocol works as follows: First, a memory strobe is initiated (`MStrobe`) that is asserted to start the memory transfer process. The `MStrobe` signal remains asserted as long as a memory transfer is initiated (i.e., a LW/SW instruction). This means you will have to modify your RISC-V RV32I microarchitecture, specifically in the control, to output this signal. Once the memory transfer is complete, it will respond with a signal (`PReady`) to indicate it is complete. After a `PReady` is received, the processor can deassert the `MStrobe` signal. This is called a handshake or asynchronous protocol. Unfortunately, your processor has its own clock and it cannot wait for the memory to complete, therefore, you have to stop or **stall** the processor to make sure it does not go forward. The easiest way to do this to use a **Enable** on your PC. Fortunately, you should have a Enabled flip-flop in your `risc_single.sv` SystemVerilog file to accomplish this task.

4.2 Synthesis

With the SystemVerilog files modified and the `.dat` imported and set up, you can use `generate bitstream` to begin processing the project. `generate bitstream` starts the toolchain that takes the project through the following steps:

Block Generation → Synthesis → Implementation → Generate Bitstream

As indicated in class, it is important that if you implement any hardware, it should be as close as possible to the given implementation using digital gates as possible. Your SystemVerilog should use a RTL or structural coding for any design. If you are unsure whether your design is RTL or structural, please ask through piazza or the instructor. Using behavioral implementations or code you find through Google searches, many result in synthesis churning for hours to find a good implementation. Do not be tempted to model your hardware incorrectly by thinking you can find a similar design on the Internet. Once implemented, make sure you test your design through thorough simulation and do not synthesis until you are sure your design will work.

If any errors occur during the toolchain execution, the process will halt and a log of the errors will be printed. More often than not, the errors explain exactly where the issue is and you can easily go to their location, solve the problem, and restart the toolchain with `generate bitstream`. The final product, the bitstream, is what you will upload to the FPGA. To do this, you will need to use the hardware manager

tool. Make sure the DSDB board is connected to the computer with the microUSB cable and powered on before trying to connect to the board. Once connected, you can program the FPGA with your bitstream. Make sure the DSDB's `SW8` switch is in *QSPI* mode while uploading your project to the board.

There is a document `fpga_guide.pdf` that documents an easy to synthesize and implement your design with the DDR3 memory controller. As mentioned in that document, it is extremely important to have your design designed and verified **before** even going to that document.

5. Lab Resources

The source code for the lab are provided through the GitHub repository as with previous labs. The source code is written in SystemVerilog, a very popular hardware description language. If you need a refresher on SystemVerilog, please refer to the Harris/Harris text as well as the Chapter 7 in the DDCA text [2]. Our textbook has some great information on this microarchitecture too in Chapters 2 and 4 [3]. As mentioned on the documents within Canvas, making your SystemVerilog resemble the digital logic you want to implement will make things easier. Do not be tempted to Google code or implement something behaviorally (i.e., caveat emptor).

6. Getting Started & Tips

1. In the top-level skeleton (`riscv_single.sv`), it already implements 13 instructions:

   ```
   add, addi, and, andi, beq, jal, lw, or, ori, slt, slti, sub, sw
   ```

   You should use these to understand how the processor works and modify the datapath/control to add the remaining instructions. Please note that LDR/STR only implements offset-based load and store instructions. Preindex and postindex types of instructions can be implemented, but might require a considerable hardware investment and/or additional cycles.

2. At this point, you should be able to simulate the test input memfile.dat by executing the following command in your shell: `vsim -do riscv_single.do`

3. You should check for correctness by comparing the register dump of the simulation against a reference register dump (e.g, through your simulator from Lab1).

4. It is far easier to add instructions one at at time and check to make sure they work before going on to the next instruction. Also, make sure you use the methodology heavily discussed in class to make things easier.

5. It is easier to implement datapath functionality first within the datapath module and then make sure it works by completing the control module afterwards.

6. You should also analyze the performance of the `test_hw.S` programs in the `riscvtest` directory in your repository with the ELVIS III board. I wrote this program to try to exercise as many RV32I instructions as possible. Address whether memory impacted or did not impact your implemented design by giving the overall speedup. There is also a collection of programs in the `testing` directory that might help test each instruction correctly. You should use programs like `spike` to help you deduce whether `test_hw.S` is working. Explain your reasoning within your lab report.

6.1 Tips

- If you have not had me before in a class, you are pretty much not going to finish this or any assignment if you start late - guaranteed [2]! Do not believe me; ask others who have had my classes. So, start early and apportion your time every day, if possible.

[2] "Luck favors the prepared!" -Louis Pasteur

- Read this handout in detail. Ask questions to the TAs or me using slack (especially early on).
- It is really easy to start with R-type instructions (e.g., xori).
- Just as a reiteration, I made the memory significantly smaller than what this RISC-V RV32I architecture can support.
- When you encounter a technical problem, please sift through your log files and look for any error specifically. Many times the error tells you the problem although not always in detail.
- Be cautious about how SystemVerilog handles signed/unsigned numbers.
- I have modified the `DO` file to segment the waveform to order the screen so it is easier to read. Try to understand what I did within the `DO` file - good clean output on a waveform can tremendously aid in your debugging experience. For example, you should see all the datapath signals in one section, whereas, the control signals are in another section. This will help you keep track of what signals are accomplishing which task. Also, I also added the memory and register file as two-dimensional arrays on the waveform within the DO file:

  ```
  add wave -hex /testbench/dut/rv32single/dp/rf/rf
  ```

  If you move your mouse over the specific signal on the waveform, you should see all values for the register file as a yellow box, as shown in Figure 2. This can help in debugging values – you can also display all the memory values by going to the Memory List window through the View Menu in Siemens Questa/ModelSim. The resulting window should appear and you can double click any memory item (i.e., two-dimensional array) to view it properly, as shown in Figure 3.

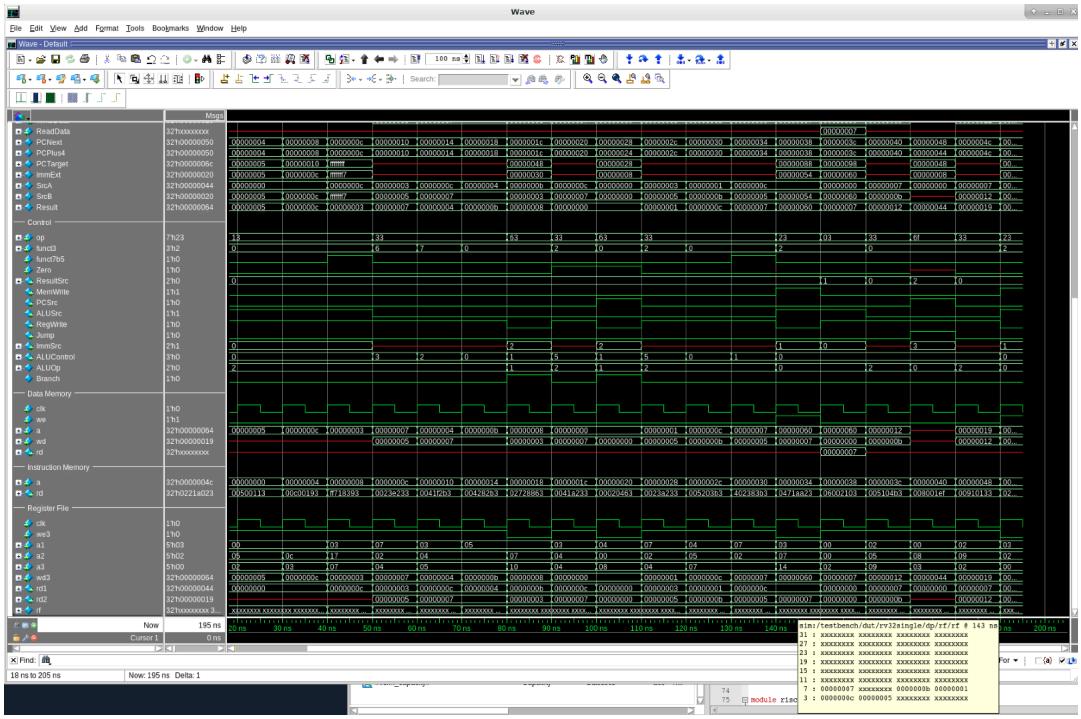

Figure 2: Two Dimensional Vector Display in Siemens Questa/ModelSim Waveform.

- Modify your `DO` file for any signals you want to identify. Sometimes, Questa/ModelSim will not log deeply-nested hierarchies unless they are identified in the `DO` file.
- Memory accesses must be aligned. If you are accessing a byte, any address is allowed. But, if you are accessing a 4-byte word, the two lowest bits of the address must be zero.

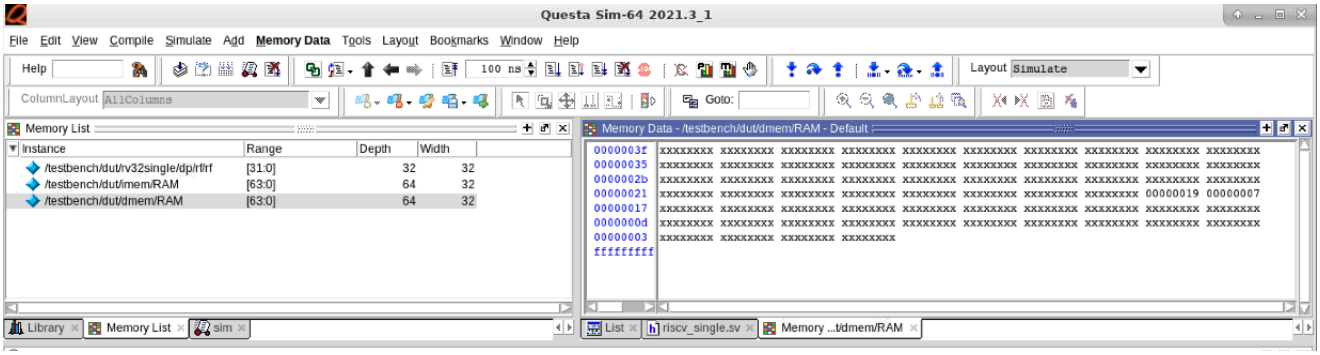

Figure 3: Memory List Display in Siemens Questa/ModelSim Waveform.

7. Handin

You should electronically hand in your code into Canvas as with Lab 1. Please contact the TAs or the instructor for more help. Your code should be readable and well-documented. In addition, please turn in additional test cases that you used in a inputs/ subdirectory. If you feel the need to describe any additional aspects of your design in detail, please include these in a separate README. Make sure that you test your Verilog extensively with a suite of test cases, just like in Lab 1, so that you are confident that you have implemented all instructions correctly. This is for your benefit in later labs!

Again, in order to grade the lab, we will use some unknown assembly code that you will not see. To get the maximum grade for this lab, your SystegmVerilog must be able to simulate all the instructions asked in Section 2. Your job is to really try to make sure your simulator covers all possible instructions that this unknown code may utilize.

8. Extra Credit

Again, there are many possibilities for extra credit, but you should only attempt this if you get the baseline project done and you have extra time.

References

[1] L. H. Crockett, R. A. Elliot, M. A. Enderwitz, and R. W. Stewart, *The Zynq Book: Embedded Processing with the Arm Cortex-A9 on the Xilinx Zynq-7000 All Programmable Soc.* UK: Strathclyde Academic Media, 2014.

[2] S. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition.* Elsevier Science, 2021.

[3] D. Harris, J. Stine, R. Thompson, and S. Harris, *RISC-V System-On-Chip Design.* Elsevier Science, 2025.